AT5 Complete Portfolio

SmartQC Product Image Quality Control System

Individual portfolio for the rookie group project. This portfolio focuses on my own work in Stage 2 image quality assessment, model comparison, ClearML tracking, product demo integration, CI/CD explanation, and final reflection.

Student
Ge Zhao
Student number: 25244738

Team
rookie
Team members: Jia Lu, Changjian Li, Ge Zhao

Project repo
AI-Studio-rookie / FinalVersion
V2 branch used for final product code

Data source
KonIQ-10k IQA Database
Used for Stage 2 MOS quality scoring

My main contribution in one sentence: I helped move SmartQC from a simple model experiment into a more complete demo workflow by building and comparing the Stage 2 image quality model, tracking experiments in ClearML, and connecting the model result to a seller-facing product demo and deployment story.

Portfolio Structure and Evidence Map

Assessment focus	Where it is covered	Main evidence
Environmental impacts	Section 2	training compute, storage, deployment impact with sources
Social impacts	Section 3	seller feedback, fairness risk, transparency and human review
Problem design and project design	Section 4	business context, ML task, datasets, requirements
POC design and implementation	Section 5	Stage 2 POC code, metrics and training curve
Interim solution V0.1	Section 6	multi-model experiments, ClearML, model comparison
Final solution V0.2	Section 7	FastAPI, MOS service, UI, Cloudflared and CI/CD design
Communicate project results	Section 8	business-facing result explanation and presentation evidence
Team reflection	Section 9	specific interactions, feelings, learning, strengths and actions
Appendix and links	Section 10	repo links, data links, code evidence map and references

Contribution scope. I mainly claim detailed ownership for Stage 2 image-quality model work, experiment comparison, ClearML evidence, and final product/demo workflow explanation. I also learned from and supported the Stage 1 and product design parts, but I do not claim that all Stage 1 core work was only done by me.

1. Project Summary

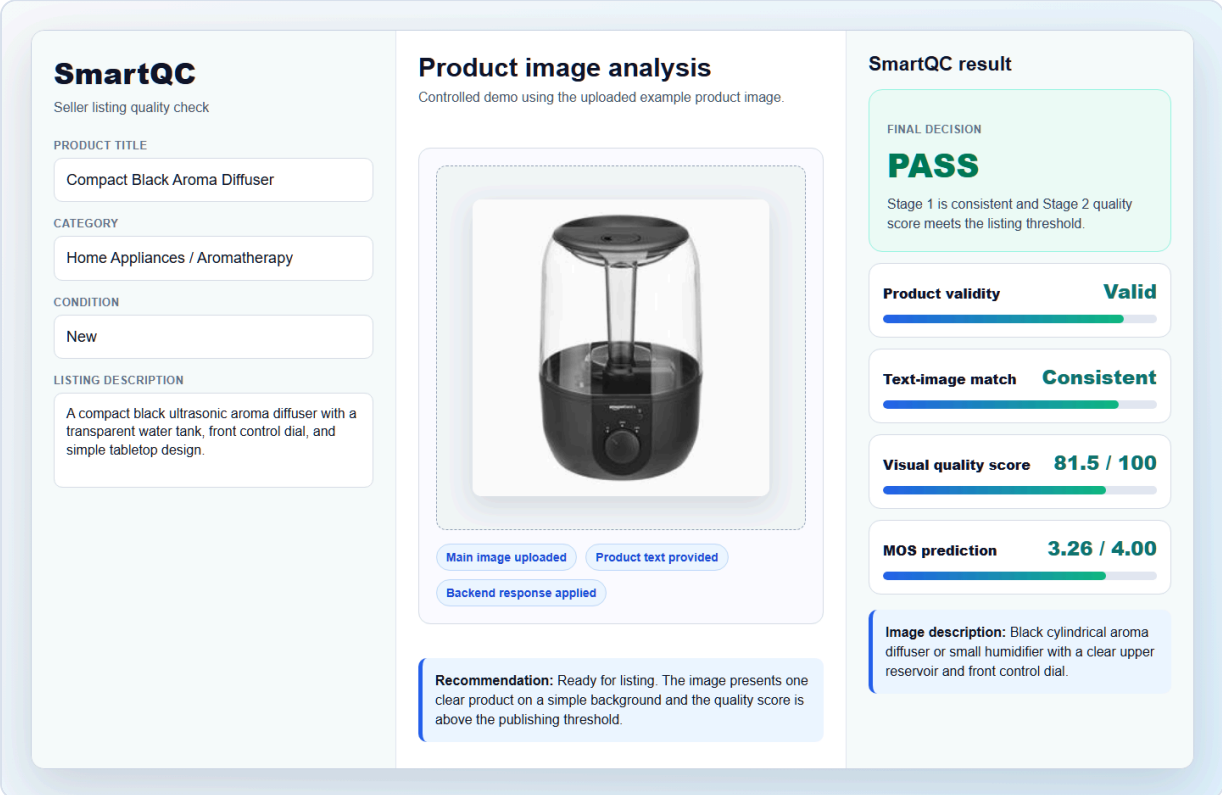
SmartQC is a two-stage AI product for e-commerce product image checking. The business problem is simple: sellers upload many product images, and manual checking is slow and not always consistent. Our idea is to give sellers a fast check before publishing, so they can fix weak images earlier.

Product workflow

- Seller uploads image and enters product text.
- Stage 1 checks product validity and text-image consistency.
- Stage 2 checks image quality if Stage 1 allows it.
- The system returns pass, review, or reject with a short reason.

My main work

- Stage 2 MOS image quality model using KonIQ-10k.
- ResNet18 and ResNet50 comparison with controlled settings.
- ClearML tracking for model runs and result comparison.
- Final demo workflow: backend model service, UI explanation, CI/CD and Cloudflared deployment notes.



Evidence 1: SmartQC V2 seller-facing demo result. The example image is uploaded, English product text is provided, and the UI shows the final recommendation and MOS quality score.

The MVP is not a full marketplace platform. It is a working demo that proves the key value: an uploaded product image can be checked by AI and converted into a clear listing recommendation.

2. Environmental Impacts

I identify three environmental impacts in the context of SmartQC. I connect them to the whole product cycle: data ingestion, model training, experiment tracking, deployment, and inference. I also include what I did or would do to reduce the impact.

Impact 1: Training compute and electricity use

Stage 2 model work used repeated training runs. I compared ResNet18 and ResNet50, different input sizes, and several settings. This used compute time and electricity. Data centres and data transmission networks are already a meaningful part of global electricity use, and AI growth makes this issue more important (International Energy Agency, 2023). For SmartQC, the main compute cost is not one inference request. The bigger cost is repeated training and experimentation.

My mitigation was to avoid random trial-and-error. I used fixed train/validation/test splits and planned comparison groups. This made the experiments more useful and reduced waste. I also used MAE and validation curves to stop chasing a larger model just because it looked stronger.

Impact 2: Storage footprint from data, models and logs

The project uses large images, CSV labels, model checkpoints, logs, screenshots and ClearML experiment records. If everything is copied into GitHub or kept forever, the storage footprint becomes messy and wasteful. This is why I separated code from large data and model files. The repository keeps code and documentation, while large data and checkpoints are referenced through external/shared storage. This is also why `.gitignore` excludes model weights, logs, uploads and raw storage folders.

Impact 3: Deployment and repeated inference cost

A deployed AI product still uses energy after training. If every listing image triggers heavy models without a reason, the product will waste compute. In SmartQC, Stage 1 acts as a gatekeeper. If an image is clearly invalid or mismatched, Stage 2 is skipped. This reduces unnecessary quality scoring. The Software Carbon Intensity idea is useful here because it encourages teams to think about emissions per software function, such as per check or per user request (Green Software Foundation, n.d.).

> **My learning:** I learned that environmental impact is not only about model training. It also appears in data storage, model versioning, logging, deployment and every future user request.

3. Social Impacts

I identify three social impacts for SmartQC. The social impact is not only technical accuracy. It is also about sellers, reviewers, transparency, fairness and trust.

Impact 1: Faster feedback for sellers and less manual review burden

The positive impact is that sellers can receive feedback earlier. Instead of waiting for manual moderation, they can see whether the image is ready, needs review, or should be retaken. This can reduce repeated back-and-forth work for sellers and platform review teams. It also makes image quality rules easier to understand.

Impact 2: Risk of unfair rejection or poor fit for some product categories

Image quality is partly subjective. A model trained on a public IQA dataset may not understand all marketplace product styles. Some product images may be creative, dark, reflective or taken in unusual backgrounds. If the model is used as a hard automatic decision, some sellers may be treated unfairly. International AI guidance stresses ideas such as fairness, transparency, accountability and human-centred AI (Organisation for Economic Co-operation and Development, 2019; UNESCO, 2021). For our product, this means SmartQC should support review instead of acting as final authority for every case.

Impact 3: Transparency and user trust

A raw ML score is hard for a seller to understand. The UI should explain the result in normal language. In V2, the system shows product validity, text-image consistency, visual quality, and a next action. NIST AI RMF also highlights trustworthiness across design, development, use and evaluation of AI systems (National Institute of Standards and Technology, 2023). In my work, I tried to connect model scores to reasons, thresholds and a recommendation, so the user can understand what to do next.

> **Important limit:** SmartQC should be treated as decision support. For real production use, it still needs human review for borderline cases, more product-category testing, and a feedback loop from real sellers.

4. Problem Design and Project Design

Business context

The project is set in an e-commerce listing workflow. The user uploads a main product image and product text. The business goal is to reduce manual checking effort, improve listing quality, and give sellers faster and clearer feedback before publishing.

ML problem involved

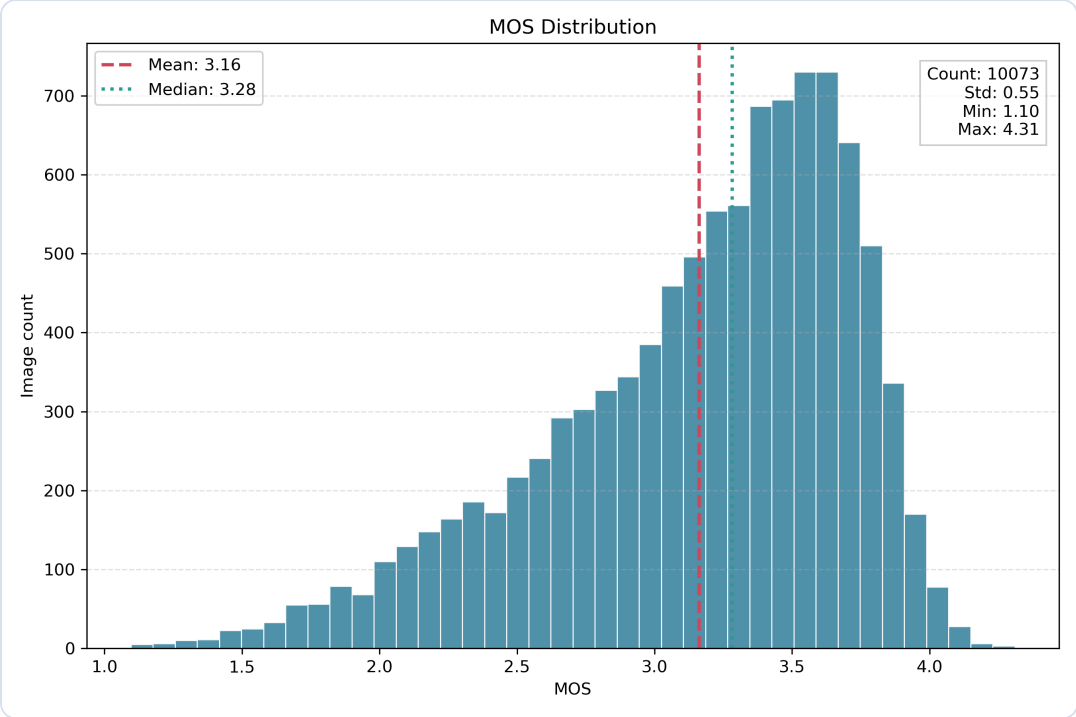
SmartQC is not only one model. It is a two-stage AI solution:

- **Stage 1:** product-image validity and text-image consistency. This is a multimodal matching / classification problem using CLIP-style embeddings.
- **Stage 2:** image quality assessment. This is a computer vision regression problem. The model predicts a quality score based on image content.
- **Final recommendation:** a rule layer converts model outputs into pass, review, or reject.

Datasets used

Stage	Dataset	Purpose	My connection
Stage 1	Amazon Berkeley Objects style product metadata and images	Create product image-text pairs and negative samples	I learned from this part and used it in the final product workflow, but I do not claim main ownership.
Stage 2	KonIQ-10k IQA Database	Train MOS image quality regression model	This is my main technical contribution area.

KonIQ-10k is a large image quality assessment database with 10,073 quality-scored images and 1.2 million crowd-sourced ratings from 1,459 workers, according to the official database page and paper (Hosu et al., 2020; Multimedia Signal Processing Group, n.d.). This made it suitable for the Stage 2 quality scoring work.



Evidence 2: MOS distribution from the KonIQ-10k based Stage 2 dataset used in my image quality model work.

My contribution to proposal and requirements

In the early project stage, I helped set up the team workspace and contributed to the two-stage idea: first check whether the image is relevant, then check quality. I also helped discuss dataset options, product feasibility, technical risks and business value. Later, I mainly developed the Stage 2 path in more depth.

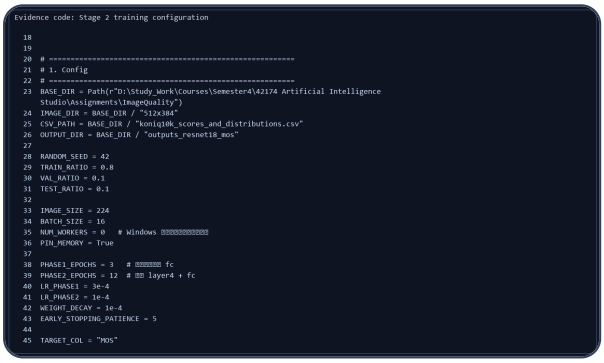
The final requirements I worked toward were practical: image upload, optional product text, quality score, plain recommendation, and demo access. I learned that requirements are not only a document. They decide what model output should look like in the UI.

5. POC Design and Implementation

The POC proved that Stage 2 image quality scoring could work. My focus was to build a ResNet-based MOS regression workflow using KonIQ-10k. I started with a relatively simple training setup, then evaluated it using clear metrics.

POC design

- Input: product or general image.
- Target: MOS quality score.
- Model: pretrained ResNet18 with a regression head.
- Output: predicted MOS, later converted into product-style quality score.
- Metrics: MAE, RMSE, PLCC and SRCC.



```
Evidence code: Stage 2 training configuration

18
19
20  # ============================================================
21  # 1. Config
22  # ============================================================
23  BASE_DIR = Path(r"D:\Study_Work\Courses\Semester4\42174 Artificial Intelligence
    Studio\Assignments\ImageQuality")
24  IMAGE_DIR = BASE_DIR / "512x384"
25  CSV_PATH = BASE_DIR / "koniq10k_scores_and_distributions.csv"
26  OUTPUT_DIR = BASE_DIR / "outputs_resnet18_mos"
27
28  RANDOM_SEED = 42
29  TRAIN_RATIO = 0.8
30  VAL_RATIO = 0.1
31  TEST_RATIO = 0.1
32
33  IMAGE_SIZE = 224
34  BATCH_SIZE = 16
35  NUM_WORKERS = 0    # Windows ■■■■■■■■■■■■
36  PIN_MEMORY = True
37
38  PHASE1_EPOCHS = 3    # ■■■■■■■ fc
39  PHASE2_EPOCHS = 12   # ■■ layer4 + fc
40  LR_PHASE1 = 3e-4
41  LR_PHASE2 = 1e-4
42  WEIGHT_DECAY = 1e-4
43  EARLY_STOPPING_PATIENCE = 5
44
45  TARGET_COL = "MOS"
```

Evidence 3: POC Stage 2 configuration. I used fixed split ratios, image size, batch size, learning rates and early stopping settings.

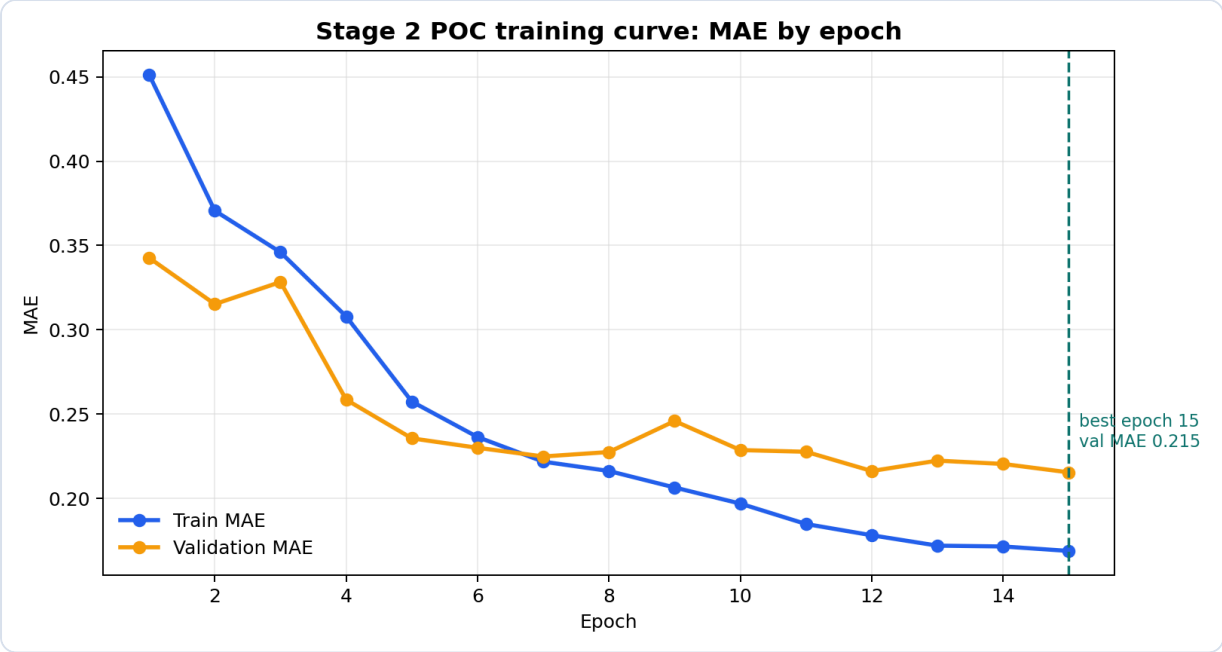
```
Evidence code: ResNet18 MOS regressor

124         if self.transform is not None:
125             image = self.transform(image)
126
127         target = torch.tensor(float(row[TARGET_COL]), dtype=torch.float32)
128         image_name = row["image_name"]
129         sd = float(row["SD"]) if "SD" in row and pd.notna(row["SD"]) else np.nan
130         c_total = float(row["c_total"]) if "c_total" in row and pd.notna(row["c_total"]) else
    np.nan
131
132         return image, target, image_name, sd, c_total
133
134
135 # ============================================================
136 # 4. Model
137 # ============================================================
138 class ResNet18MOSRegressor(nn.Module):
139     """
140     ■■■■■ sigmoid ■■■■ [1, 5]
141     """
142     def __init__(self):
143         super().__init__()
144         weights = ResNet18_Weights.DEFAULT
145         self.backbone = models.resnet18(weights=weights)
146         in_features = self.backbone.fc.in_features
147         self.backbone.fc = nn.Sequential(
148             nn.Dropout(p=0.2),
149             nn.Linear(in_features, 1)
150         )
151
152     def forward(self, x):
153         raw = self.backbone(x).squeeze(1)
```

Evidence 4: POC ResNet18 MOS regressor. The classification head is changed into a regression head and output is mapped to MOS range.

POC implementation quality

I did not only train a model once. I saved train/validation/test splits, checkpoints, training logs and test outputs. This helped the POC become repeatable. The test summary showed:

Metric	Value from test_summary.json	Meaning
Best epoch	15	The best validation result was selected by evidence.
Test MAE	0.2097	Average MOS error was around 0.21 on the test set.
Test RMSE	0.2699	Shows overall error size.
PLCC / SRCC	0.875 / 0.843	The predictions had strong linear and rank correlation with true MOS.



Evidence 5: POC training curve generated from training_log.csv. It shows model improvement and the best validation point.

Evidence code: model evaluation metrics

```
    if len(y_true) < 2:
        return float("nan")
    if np.std(y_true) == 0 or np.std(y_pred) == 0:
        return float("nan")
    return float(np.corrcoef(y_true, y_pred)[0, 1])


def safe_spearmanr(y_true, y_pred):
    y_true = pd.Series(y_true).rank(method="average").to_numpy()
    y_pred = pd.Series(y_pred).rank(method="average").to_numpy()
    return safe_pearsonr(y_true, y_pred)


def compute_metrics(y_true, y_pred):
    y_true = np.asarray(y_true, dtype=np.float32)
    y_pred = np.asarray(y_pred, dtype=np.float32)

    mae = mean_absolute_error(y_true, y_pred)
    rmse = math.sqrt(mean_squared_error(y_true, y_pred))
    plcc = safe_pearsonr(y_true, y_pred)
    srcc = safe_spearmanr(y_true, y_pred)

    return {
        "mae": float(mae),
        "rmse": float(rmse),
        "plcc": float(plcc) if not np.isnan(plcc) else float("nan"),
```

Evidence 6: Evaluation code for MAE, RMSE, PLCC and SRCC. I used metrics that can explain both error and ranking quality.

From this POC, I learned that a model result is stronger when it is saved with configs, splits, logs and test predictions. Without those files, the score is hard to trust later.

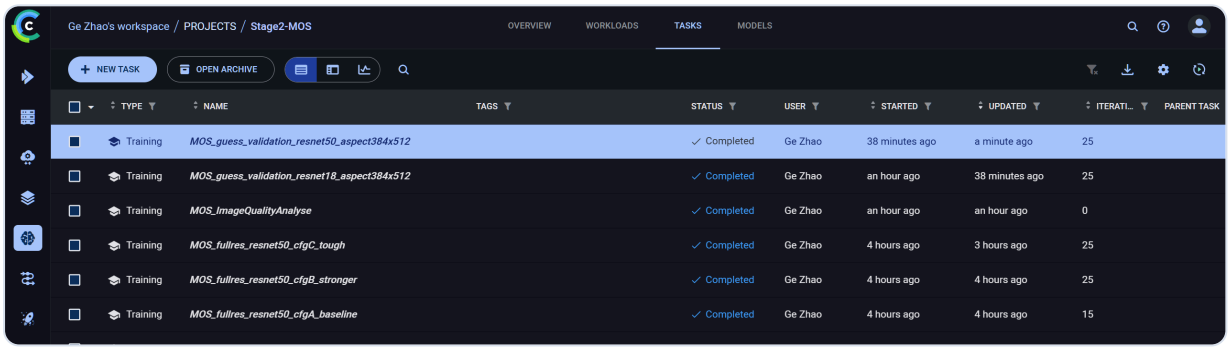
6. Interim Solution V0.1

In the interim solution, I moved Stage 2 from a small POC mindset to a more systematic model development process. The main change was controlled experiment comparison and ClearML tracking.

V0.1 design

I designed the comparison around two CNN backbones and multiple training settings. I wanted to know whether a deeper model was always better, or whether input size and preprocessing mattered more. This was important because product image quality depends on details such as blur, exposure and object clarity.

- Backbones: ResNet18 and ResNet50.
- Settings: baseline, stronger, tough, and larger aspect-ratio-preserving input.
- Tracking: ClearML tasks for configurations, curves, logs and model version evidence.
- Main metric: MAE, because it directly means average score error.

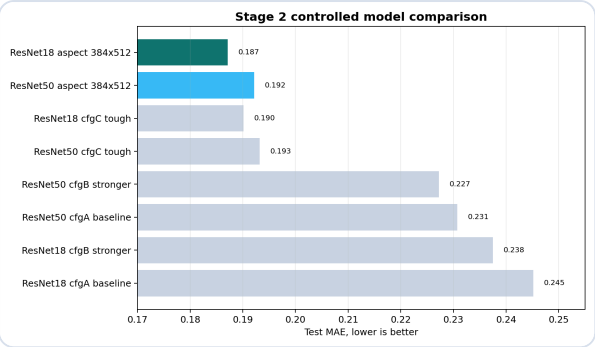


Evidence 7: ClearML task list. Multiple Stage 2 training runs were tracked separately instead of using manual notes only.

Evidence 8: ClearML scalar curves. I used these curves to compare loss, MSE and resource behaviour during training.

What I found

The result was not exactly what I first expected. I thought ResNet50 would clearly win because it is deeper. But the comparison showed that input setting and aspect-ratio preservation were very important. The best result in the comparison table was ResNet18 with aspect 384x512, while ResNet50 aspect 384x512 was close. This made me think more carefully about preprocessing, not only model depth.



Evidence 9: Model selection chart. Lower test MAE is better; aspect-aware inputs performed strongly.

model_name	config_name	input_size	batch_size	best_epoch	best_val_mse	test_mse	test_mae	test_rmse
resnet50	guess_validation_biginput	384x512	6	24	0.065448	0.062016	0.192162	0.249030
resnet18	guess_validation_biginput	384x512	6	23	0.065666	0.058742	0.187145	0.242368
resnet18	cfgC_tough	384	6	18	0.066487	0.061968	0.190103	0.248935
resnet50	cfgC_tough	384	6	20	0.067588	0.063417	0.193209	0.251827
resnet50	cfgA_baseline	224	16	14	0.094028	0.092080	0.230748	0.303446
resnet18	cfgB_stronger	224	16	22	0.096031	0.092881	0.237516	0.304763
resnet50	cfgB_stronger	224	16	18	0.096725	0.086829	0.227225	0.294668
resnet18	cfgA_baseline	224	16	14	0.105775	0.099513	0.245135	0.315458

Evidence 10: Model comparison table screenshot used in presentation and portfolio evidence.

> **My learning from V0.1:** A bigger model is not automatically a better product model. A fair split, clear metric, and careful preprocessing can matter more than simply choosing a deeper backbone.

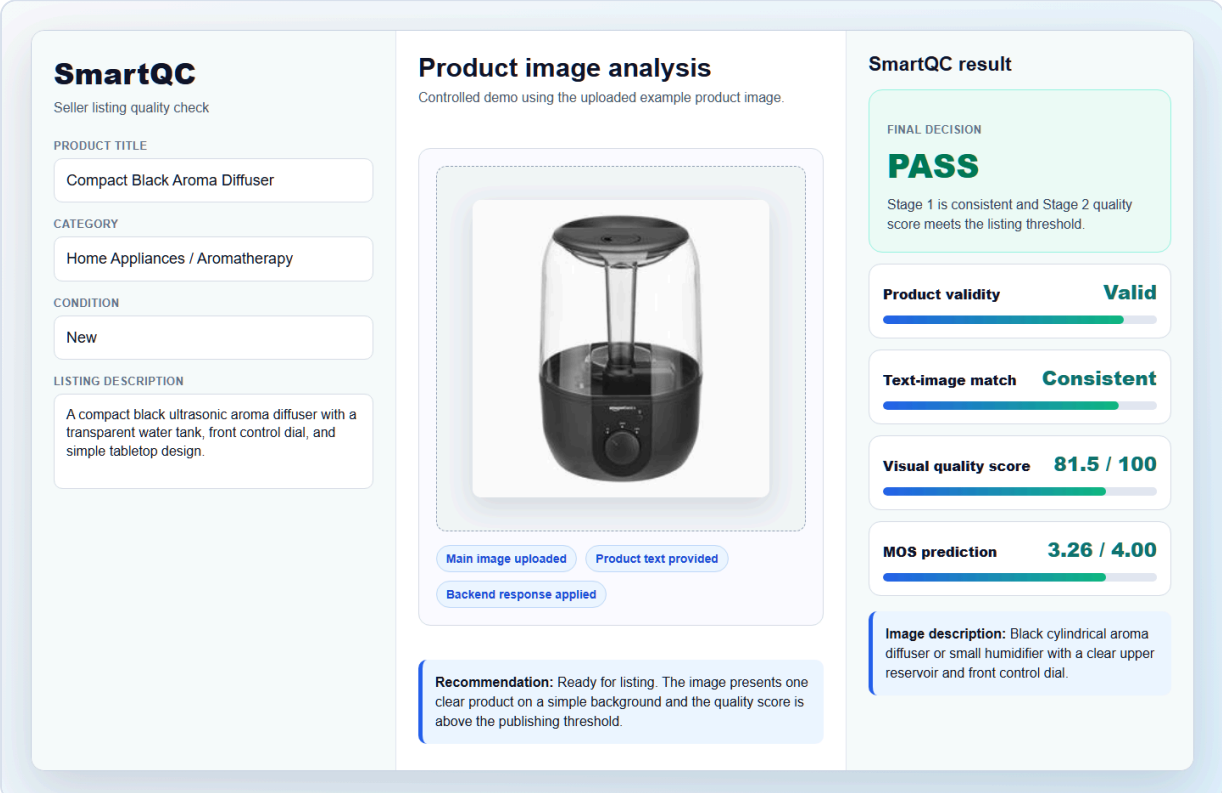
7. Final Solution V0.2

The final solution connected separate model work into a working product demo. This is where the project started to feel less like a notebook and more like an AI product.

V0.2 design

The final design has a frontend, a FastAPI backend, two AI services, a rule-based listing decision layer, draft storage and demo deployment support.

Component	Purpose	Evidence
Static frontend	Seller listing form, image upload, SmartQC result panel	index.html, app.js, styles.css
FastAPI backend	Receives image and text, calls Stage 1 and Stage 2 services	backend/main.py
Stage 1 service	CLIP product validity and text-image consistency check	backend/clip_service.py
Stage 2 service	Loads MOS model and predicts quality score	backend/mos_service.py
Draft store	Saves demo listing records and uploaded image files	backend/draft_service.py
Deployment path	Vercel frontend, Python backend, Cloudflared public demo link	README, vercel.json, Cloudflared slide



Evidence 11: Final SmartQC UI in V0.2. The result screen shows product validity, text-image match, visual quality score, MOS prediction and listing decision.

Implementation evidence

The backend decides whether Stage 2 should run. If Stage 1 detects a clearly invalid image or a mismatch, Stage 2 is skipped. This saves compute and gives a clear reason to the user. If the image passes Stage 1, Stage 2 predicts a quality score. Then the final rule returns pass, review or reject.

Evidence code: two-stage skip and listing decision logic

```python
def should_skip_stage2(stage1: Dict[str, Any]) -> bool:
    status = stage1.get("status")
    invalid_score = stage1.get("invalid_score")
    final_score = stage1.get("final_score")

    if status in {"inconsistent", "invalid_image"}:
        return True
    if invalid_score is not None and invalid_score >= clip_service.invalid_score_threshold + 0.05:
        return True
    if final_score is not None and final_score <= clip_service.negative_threshold:
        return True
    return False


def skipped_stage2_result() -> Dict[str, Any]:
    return {
        "skipped": True,
        "skipped_reason": STAGE2_SKIPPED_MESSAGE,
        "quality_score": None,
        "mos_score": None,
        "message": STAGE2_SKIPPED_MESSAGE,
    }


def stage1_indicates_fatal_failure(stage1: Dict[str, Any]) -> bool:
    invalid_score = stage1.get("invalid_score")
    final_score = stage1.get("final_score")
    return (
        stage1.get("status") in {"invalid_image", "inconsistent"}
        or (invalid_score is not None and invalid_score >= clip_service.invalid_score_threshold + 0.05)
        or (final_score is not None and final_score <= clip_service.negative_threshold)
    )


def decide_listing(stage1: Dict[str, Any], stage2: Dict[str, Any]) -> str:
    status = stage1.get("status")

    if status in {"invalid_image", "inconsistent"}:
        return "reject"
    if stage2.get("skipped"):
        return "reject" if stage1_indicates_fatal_failure(stage1) else "review"
    if status in {"review", "missing_text_review"}:
        return "review"
    if status == "consistent":
        quality_score = stage2.get("quality_score")
        if quality_score is None:
            return "review"
        if quality_score >= 70:
            return "pass"
        if quality_score >= 50:
            return "review"
        return "reject"
    return "review"
```

Evidence 12: Backend two-stage decision logic. It connects Stage 1, Stage 2 and final listing recommendation.

Evidence code: deployed MOS preprocessing and prediction

```
        x = torch.flatten(x, 1)
        x = self.fc(x)
        return x


def build_model(model_name: str) -> nn.Module:
    if model_name == "resnet18":
        return ResNet(BasicBlock, [2, 2, 2, 2], num_outputs=1)
    elif model_name == "resnet50":
        return ResNet(Bottleneck, [3, 4, 6, 3], num_outputs=1)
    else:
        raise ValueError(f"Unsupported MOS model name: {model_name}")


def preprocess_image(image: Image.Image) -> torch.Tensor:
    image = image.convert("RGB")
    image = image.resize((MOS_INPUT_WIDTH, MOS_INPUT_HEIGHT))
    array = np.asarray(image).astype(np.float32) / 255.0
    array = (array - IMAGENET_MEAN) / IMAGENET_STD
    array = np.transpose(array, (2, 0, 1))
    return torch.from_numpy(array).unsqueeze(0)


def mos_to_100(mos_score: float) -> float:
    normalized_score = ((mos_score - MOS_MIN_SCORE) / MOS_NORMALIZATION_INTERVAL) * 100.0
    return float(np.clip(normalized_score, 0.0, 100.0))


class MosService:
    def __init__(
        self,
        model_path: Path = MOS_MODEL_PATH,
        model_name: str = MOS_MODEL_NAME,
    ) -> None:
        self.model_path = Path(model_path)
        self.model_name = model_name
        self.device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
        self.model: nn.Module | None = None

    def load(self) -> None:
        if self.model is not None:
            return

        if not self.model_path.exists():
            raise FileNotFoundError(f"MOS model file not found: {self.model_path}")

        model = build_model(self.model_name).to(self.device)
        checkpoint = torch.load(self.model_path, map_location=self.device)
        state_dict = checkpoint.get("state_dict", checkpoint) if isinstance(checkpoint,
    dict) else checkpoint
        model.load_state_dict(state_dict)
        model.eval()
        self.model = model

    @torch.no_grad()
    def predict(self, image: Image.Image) -> MosPrediction:
        self.load()
        assert self.model is not None

        tensor = preprocess_image(image).to(self.device)
```

Evidence 13: Final MOS service. The saved model checkpoint is loaded and the MOS result is converted to a 0-100 quality score.

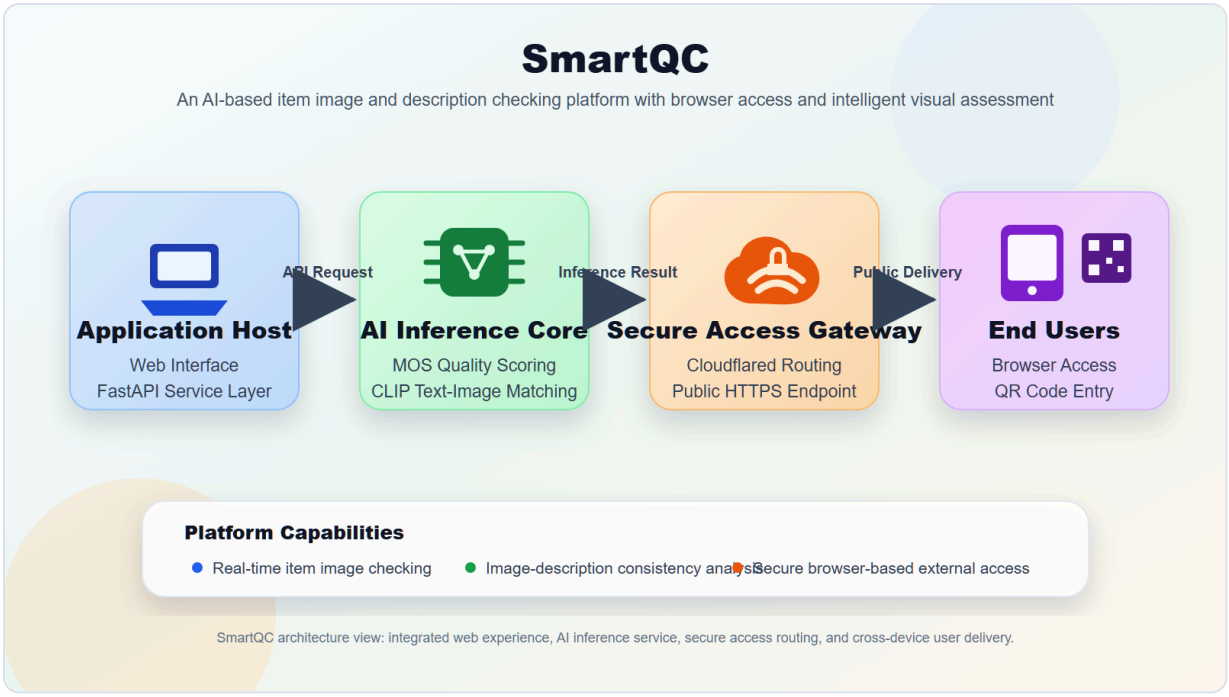
Evidence code: frontend /analyze API request

```
async function analyze() {
  if (!state.file) {
    showMessage("Please upload a main product image first.");
    return;
  }

  const apiBase = getApiBase();
  setBusy(true, "Checking listing...");
  showMessage("");
  setAssistantProgress("Running Stage 1 CLIP gatekeeper...");

  if (!hasProductText()) showMessage(NO_TEXT_NOTICE, "notice");

  try {
    const result = await callBackend(apiBase);
    setBusy(true, "Generating final decision...");

    if (result?.success === false) {
      showModelError(result, apiBase);
      return;
    }

    applyBackendResult(result);
    setBackendState("connected", "Model API ready", "Connected");
  } catch (error) {
    showRequestError(error, apiBase);
  } finally {
    setBusy(false);
  }
}

async function callBackend(apiBase) {
  const formData = new FormData();
  formData.append("image", state.file);
  formData.append("product_name", elements.productNameInput.value.trim());
  formData.append("description", elements.descriptionInput.value.trim());

  const response = await fetch(`${apiBase}/analyze`, {
    method: "POST",
    body: formData,
  });

  const contentType = response.headers.get("content-type") || "";
  const payload = contentType.includes("application/json")
    ? await response.json()
    : await response.text();

  if (!response.ok) {
    const detail = typeof payload === "string" ? payload : payload?.detail ||
JSON.stringify(payload);
    throw new Error(`Backend returned ${response.status}: ${detail}`);
  }

```

Evidence 14: Frontend API call. The UI sends image and product text to /analyze and then applies the result to the page.

CI/CD and deployment design

The current repository has Vercel frontend configuration and backend run instructions. I also prepared CI/CD workflow explanation for the final presentation. The intended GitHub Actions workflow is to check dependency installation, basic import/runtime checks, model file policy, and deployment readiness. Secrets such as ClearML keys or deployment tokens should go into GitHub Secrets, not into repo files.



Evidence 15: Cloudflared workflow diagram. It explains how a temporary SmartQC demo can be exposed through a public URL.

Cloudflared is not full production deployment. I treat it as a demo access method for tutors and teammates to test the product quickly.

8. Communicating Project Results

I tried to communicate the project result in a way that a business audience can understand. I did not only say "the model has a lower MAE". I connected the model result to seller value, platform review efficiency, and product readiness.

Business-facing result story

For sellers

They get quick feedback before publishing. The output is pass, review or reject, not only a raw model number.

For platform reviewers

The system can reduce obvious invalid or low-quality submissions and make manual review more focused.

For future product work

ClearML and comparison results make model improvement more traceable, instead of relying on memory or one notebook.

How I presented my part

- I explained Stage 2 from dataset, model, experiment comparison, final model, and product output.
- I used ClearML screenshots and metric tables so the audience could see real evidence.
- I explained Cloudflared and CI/CD as productisation support, not just extra tools.
- I used simple language in slides because the project should be understood by business users and technical users.

> **Claimed level for communication of project results:** Level 4. My evidence is the structured presentation of data, model comparison, product UI, deployment workflow and business value. I used charts, screenshots and code evidence instead of only writing claims.

9. Reflection on Team Interactions

This section is based on my four reflective journals and my final project experience. I write it in first person because it is about how the project team influenced me and how I influenced the team.

Positive interactions

One positive interaction was the early discussion about project direction. I put forward the two-stage pipeline idea, and the team discussed whether it made sense for product image checking. This helped me learn that a good AI project needs both technical feasibility and business meaning. My teammates also helped me think more about how a seller or reviewer would understand the result.

Another positive interaction happened during the later presentation work. When I prepared Stage 2, ClearML, Cloudflared and CI/CD content, teammates asked questions about how to explain them more simply. This pushed me to translate technical words into a product story.

Challenging interactions

The main challenge was progress pressure near the final deadline. Some parts were slower than I expected, and I felt anxious because the model, UI, deployment story and final documents had to connect together. I sometimes pushed teammates directly. This helped the project move, but I also know it could make the communication feel stressful.

Another challenge was that I sometimes worked on technical problems for too long by myself before explaining the uncertainty to others. For example, when model comparison results did not match my first assumption, I needed time to change my thinking from "ResNet50 should be better" to "the result is telling me preprocessing matters".

My role, feelings, strengths and weakness

Area	Reflection
My role	I acted as a technical contributor and also a progress pusher. My main technical role was Stage 2 model work and experiment tracking.
Feelings	I felt confident when technical work was moving, but anxious when deadlines were close and some parts were not connected.
Strength	I can turn a vague technical idea into runnable experiments and evidence. I am willing to take responsibility.
Weakness	I can become impatient under pressure. I also sometimes explain decisions too fast because I already understand the background.

How the team influenced me

My teammates influenced me by reminding me that the project is not only about model performance. They made me think about presentation, UI, business value and simple explanation. This changed how I wrote slides and how I explained results. I learned that a model is only useful if the user can understand the output and trust the workflow.

How I influenced the team

I think my influence was mainly technical structure and momentum. I helped turn Stage 2 from one POC into a more complete experiment process. I also pushed the team to connect the project into a final demo story: data, model, UI, deployment and product value. This made the final project easier to present as one product, not several separate files.

Actions I will take next time

- Start deployment planning earlier, not after model training is nearly finished.
- Give shorter and earlier progress updates when I am unsure about a technical assumption.
- Use a task list and simple deadlines instead of pushing only when the deadline is close.
- Explain technical decisions with examples, not only model names and metrics.
- Ask teammates what support they need before a task becomes late.
- Keep using experiment tracking tools when there are many runs.

> **Claimed level for team reflection:** Level 4. I described specific positive and challenging interactions, my feelings, my strengths and weaknesses, how others influenced me, how I influenced the team, and concrete actions for future improvement.

10. Claimed Performance Levels

This table summarises my claimed performance levels against the AT5 rubric. I keep the claims connected to evidence in this report.

Rubric criterion	Claimed level	Short reason
Environmental impacts	Level 4	I identify three impacts across training, storage and deployment, with sources and mitigation.
Social impacts	Level 4	I identify seller benefit, unfair rejection risk, and transparency/human review need, with sources.
Problem design	Level 4	The use case is a real two-stage product workflow using multimodal matching and IQA regression.
Project design proposal	Level 4	I contributed to the two-stage product idea and helped shape requirements around product image checking and quality scoring.
Design of POC	Level 4	I designed Stage 2 POC around KonIQ-10k, MOS regression, metrics and repeatable splits.
Design of interim solution V0.1	Level 4	I designed controlled experiments and ClearML tracking for multi-model comparison.
Design of final solution V0.2	Level 4	I helped connect Stage 2 output with backend logic, UI, deployment and product recommendation.
Implementation of POC	Level 4	Code evidence shows training config, ResNet model, metrics, logs and saved outputs.
Implementation of interim solution V0.1	Level 4	Evidence includes ClearML screenshots, model comparison table and controlled run results.
Implementation of final solution V0.2	Level 4	Evidence includes backend MOS service, decision logic, frontend API call and UI screenshot.
Communicate project results	Level 4	I present a structured product story with visuals and business meaning.
Communication skills	Level 3	I communicated actively and clearly, but I still need to improve tone and patience under pressure.
Reflection on team interactions	Level 4	I give detailed self-reflection, team influence, learning evidence, and future actions.

Appendix A: Extra Evidence Screenshots

Evidence code: Stage 1 prompt banks and thresholds

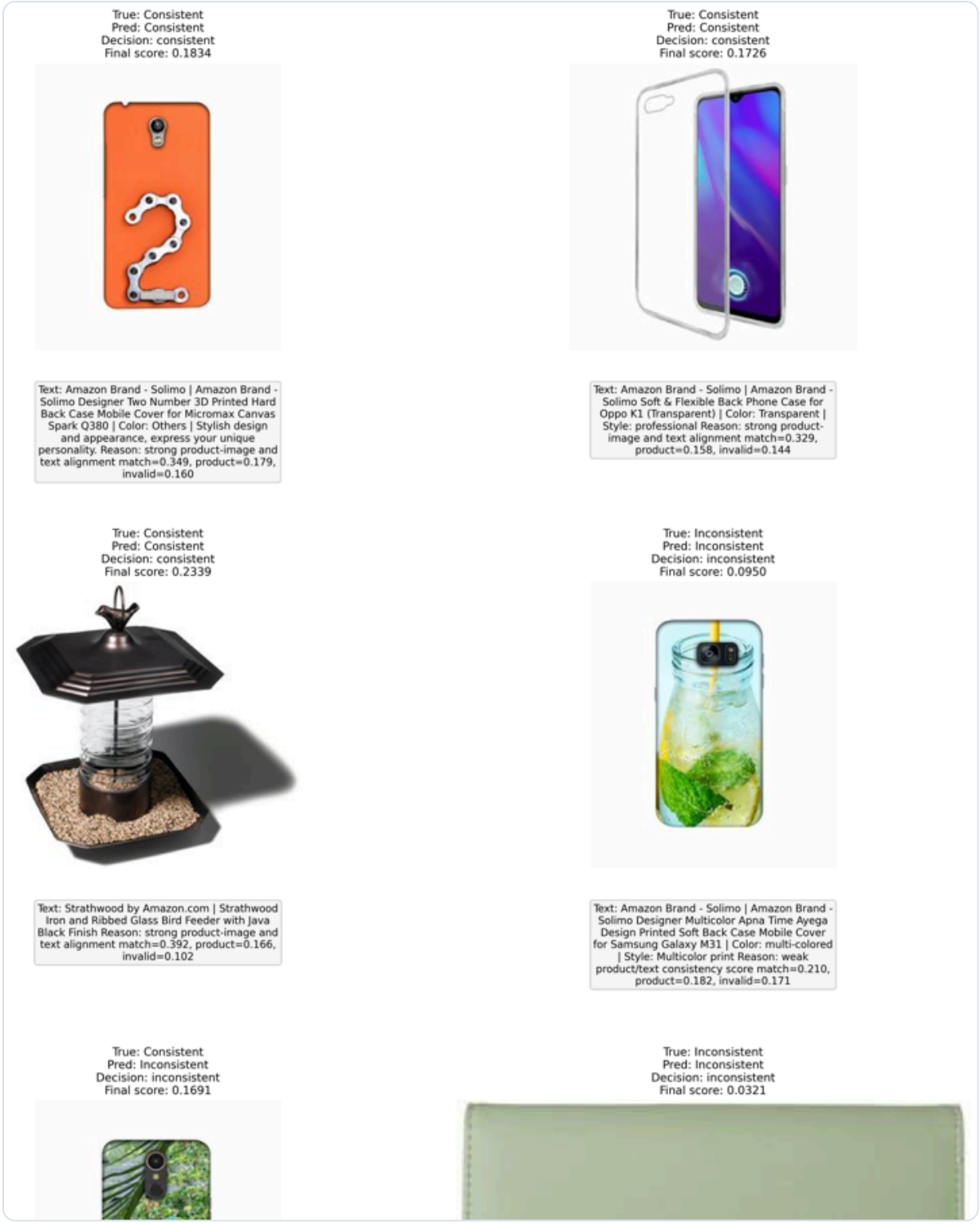
```
from .config import CLIP_BACKEND, CLIP_HF_MODEL_ID, CLIP_MODEL_NAME, CLIP_PRETRAINED


PRODUCTNESS_PROMPTS = [
    "a clean e-commerce product photo",
    "a product listing image with one main product",
    "a retail product image on a simple background",
    "a clear product-focused catalog photo",
]

INVALID_IMAGE_PROMPTS = [
    "a screenshot of a shopping app or webpage",
    "a promotional poster with large text",
    "a collage of multiple images",
    "a selfie or unrelated person photo",
    "a cluttered scene with no clear main product",
    "a marketing banner or advertisement",
]


# Risk-tolerant Stage 1 gatekeeper thresholds.
# Stage 1 should reject obvious invalid/mismatched images, while allowing
# visually valid but lower-quality product photos to continue to Stage 2 MOS.
DEFAULT_POSITIVE_THRESHOLD = 0.22
DEFAULT_NEGATIVE_THRESHOLD = 0.15
DEFAULT_PRODUCT_MARGIN_THRESHOLD = 0.00
DEFAULT_INVALID_SCORE_THRESHOLD = 0.28
```

Appendix Evidence A1: Stage 1 prompt banks and threshold settings used in the final backend service.

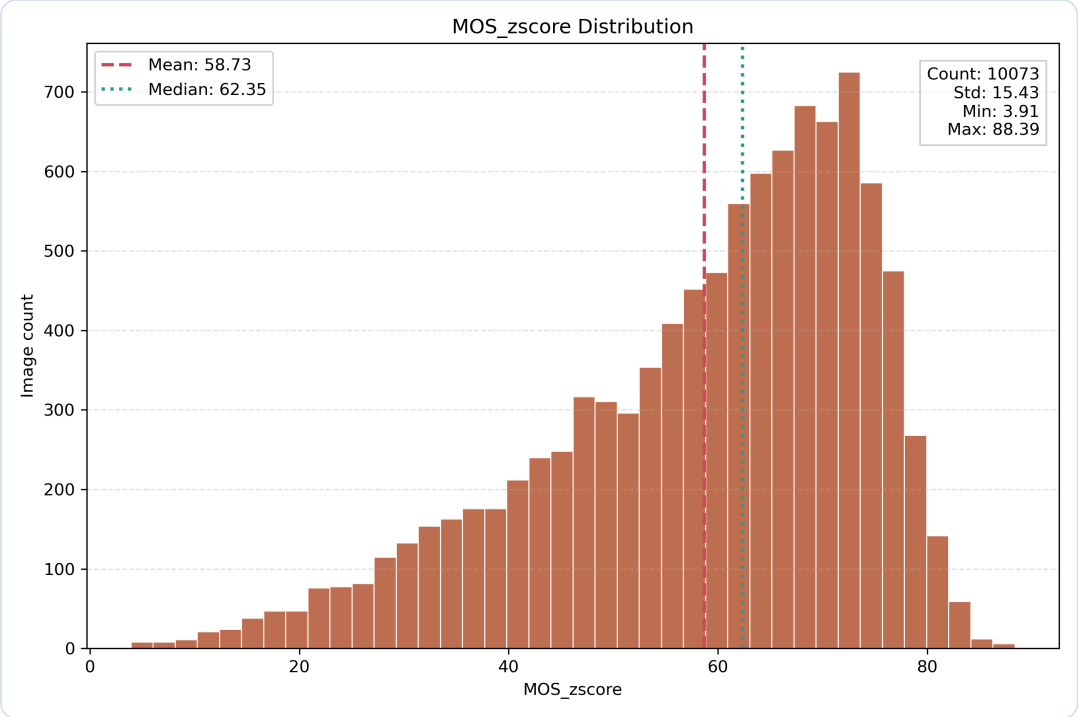
Evidence code: ABO product text feature preparation

```

def choose_language_value(
    values: Sequence,
    language_priority: Sequence[str],
) -> Optional[str]:
    entries = ensure_list(values)
    normalized_entries: List[Dict] = []

    for entry in entries:
        entry = safe_literal(entry)
        if isinstance(entry, dict):
            normalized_entries.append(entry)

    if not normalized_entries:
        return None

    for language_tag in language_priority:
        for entry in normalized_entries:
            value = str(entry.get("value", "")).strip()
            if entry.get("language_tag") == language_tag and value and
is_english_like(value):
                return value

    return None


def choose_plain_values(values: Sequence, language_priority: Sequence[str]) -> List[str]:
    entries = ensure_list(values)
    selected: List[str] = []

    for entry in entries:
        entry = safe_literal(entry)
        if isinstance(entry, dict):
            value = str(entry.get("value", "")).strip()
            language_tag = entry.get("language_tag")
            if value and language_tag in language_priority and is_english_like(value):
                selected.append(value)
        elif isinstance(entry, str) and entry.strip() and is_english_like(entry.strip()):
            selected.append(entry.strip())

    unique_values = list(dict.fromkeys(selected))
    return unique_values


def build_text_description(item: Dict, language_priority: Sequence[str]) -> Optional[str]:
    """
    Build a compact product description from ABO listing metadata.
```

Appendix Evidence A2: ABO product text feature preparation for Stage 1 image-text consistency data.



Appendix Evidence A3: Stage 1 sample predictions and score visualisation from the project folder.



Appendix Evidence A4: Additional KonIQ distribution evidence used during Stage 2 dataset understanding.

Appendix B: Links and File Map

Important links

- V2 product code repo: https://github.com/jiaLu4/AI-Studio-rookie/tree/FinalVersion
- Portfolio evidence folder: https://github.com/jiaLu4/AI-Studio-rookie/tree/main/Ge_Zhao_Portfolio
- Project Confluence documentation: https://ai-studio-rookie.atlassian.net/wiki/spaces/A/overview?homepageId=163949
- Personal reflective journals: https://ai-studio-rookie.atlassian.net/wiki/spaces/A/pages/3276856/Reflective+Journals_Ge+Zhao
- KonIQ-10k data source: https://database.mmsp-kn.de/koniq-10k-database.html

Online Evidence Files Used

Evidence area	Online evidence links
Stage 2 POC	training configuration screenshot, ResNet MOS model screenshot, evaluation metric screenshot, training MAE curve
Model comparison	ClearML task screenshot, ClearML scalar screenshot, model selection chart, model comparison table
Final backend	two-stage backend decision screenshot, MOS service screenshot, Stage 1 prompt and threshold screenshot
Frontend UI	English SmartQC GUI result screenshot, frontend API call screenshot
Deployment	Cloudflared workflow diagram, portfolio evidence README
Reflection	personal reflective journal page in Confluence, team Confluence documentation space

References

Green Software Foundation. (n.d.). *Software Carbon Intensity*. Retrieved May 24, 2026, from https://greensoftware.foundation/standards/sci/

Hosu, V., Lin, H., Sziranyi, T., & Saupe, D. (2020). KonIQ-10k: An ecologically valid database for deep learning of blind image quality assessment. *IEEE Transactions on Image Processing, 29*, 4041-4056. https://doi.org/10.1109/TIP.2020.2967829

International Energy Agency. (2023, July 11). *Data centres and data transmission networks*. https://www.iea.org/energy-system/digitalisation/data-centres-and-data-transmission-networks

Multimedia Signal Processing Group. (n.d.). *KonIQ-10k IQA database*. Retrieved May 24, 2026, from https://database.mmsp-kn.de/koniq-10k-database.html

National Institute of Standards and Technology. (2023, January 26). *AI risk management framework*. https://www.nist.gov/itl/ai-risk-management-framework

Organisation for Economic Co-operation and Development. (2019). *OECD AI principles*. https://www.oecd.org/en/topics/ai-principles.html

UNESCO. (2021). *Recommendation on the ethics of artificial intelligence*. https://www.unesco.org/en/articles/recommendation-ethics-artificial-intelligence